CEUB

EDUCAÇÃO SUPERIOR

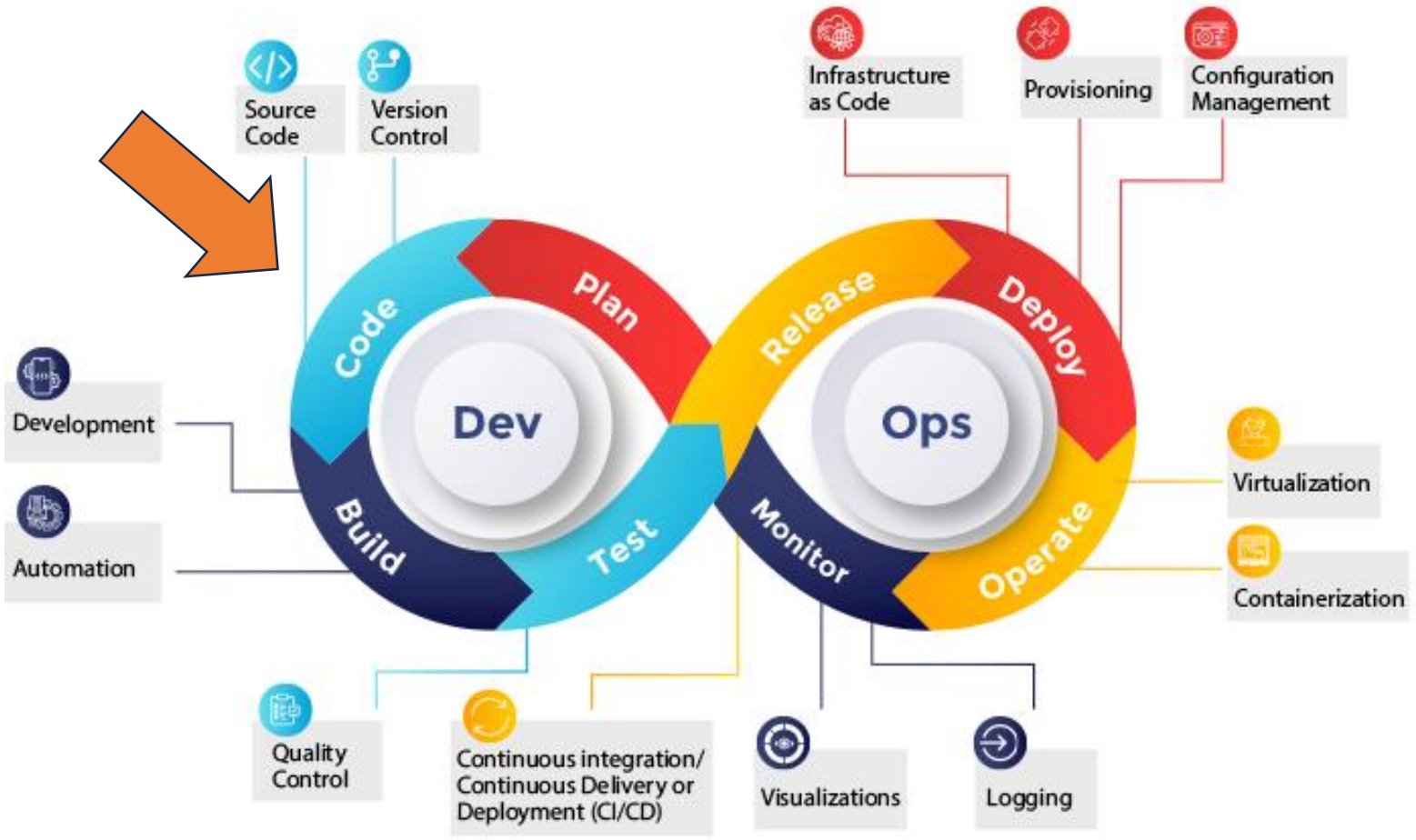
# Versionamento de Código e Colaboração com Git

## Aula 3

**Professor Danilo Silva**

ceub.br

# Onde estamos?



# DevOps

## Fundamentos do Controle de Versão

- **Arquiteturas de Versionamento:**
  - **Centralizado (CVCS - ex: SVN, CVS):** Dependência de um servidor central para operações e histórico; ponto único de falha.
  - **Distribuído (DVCS - ex: Git, Mercurial):** Todo desenvolvedor possui uma cópia completa do histórico e do repositório localmente.

CEUB



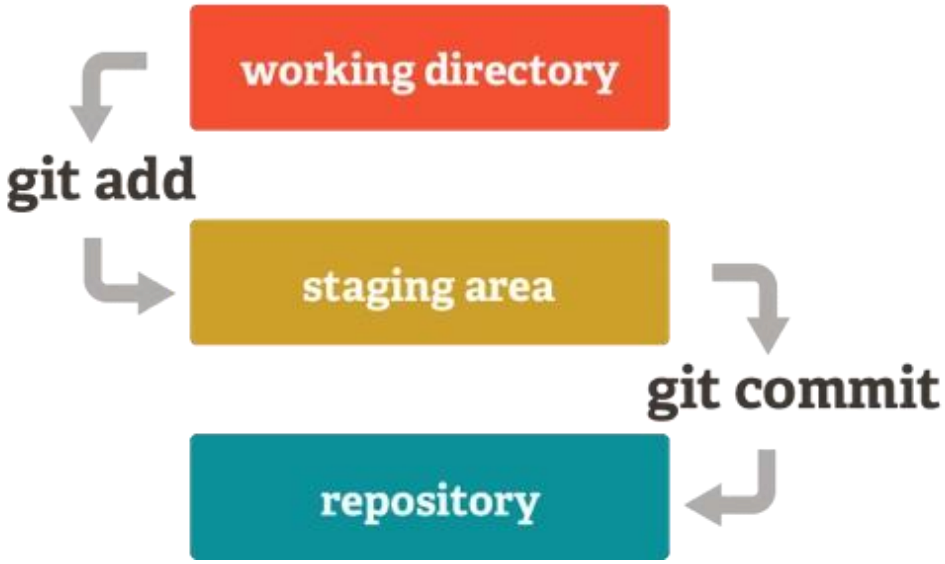

## O que é Git?

- O **Git** é um sistema de controle de versão distribuído, usado para rastrear alterações em arquivos e códigos-fonte durante o desenvolvimento de software.
- Criado por Linus Torvalds em 2005, ele permite que equipes registrem o histórico de modificações e trabalhem em conjunto de forma organizada.

CEUB

# Estados e Ciclo de vida do Código no Git

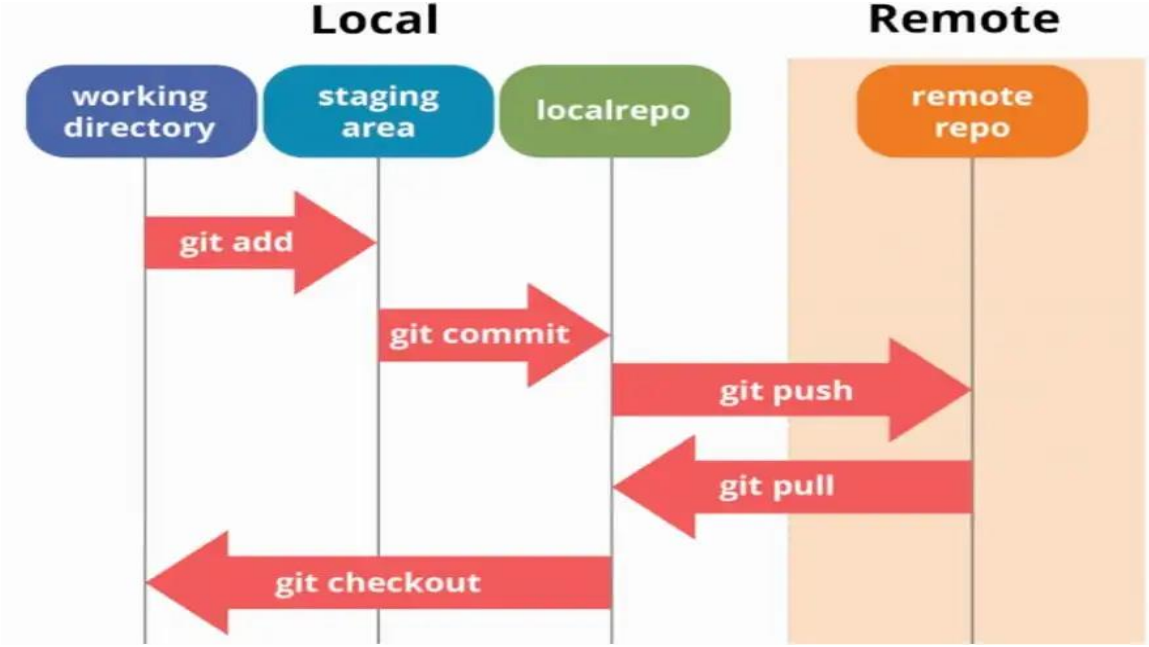
- As três áreas principais: **Working Directory** (diretório de trabalho), **Staging Area / Index** (zona de preparação) e **Repository / Local Database** (histórico commitado).
- Estados dos arquivos: Untracked, Unmodified, Modified e Staged.




| | |
|---|---|
| Untracked | *Arquivo novo e não rastreável dentro do diretório monitorado pelo git.* |
| Tracked | *Arquivo rastreável adicionado no diretório monitorado pelo git.* |
| Modified | *Arquivo alterado no diretório monitorado pelo git.* |
| Staged | *Arquivo finalizado no diretório monitorado pelo git.* |

# Operações Fundamentais e Fluxo de Trabalho:

- **Criação e Registro:** *init*, *clone*, *add*, *commit* (garantindo rastreabilidade com autor, data e mensagem).
- **Inspeção e Histórico:** *status*, *log*, *diff* e *blame* (identificando quem alterou qual linha e quando).
- **Navegação e Descarte:** *checkout/switch*, *reset*, *revert* e *stash* (armazenamento temporário de alterações).





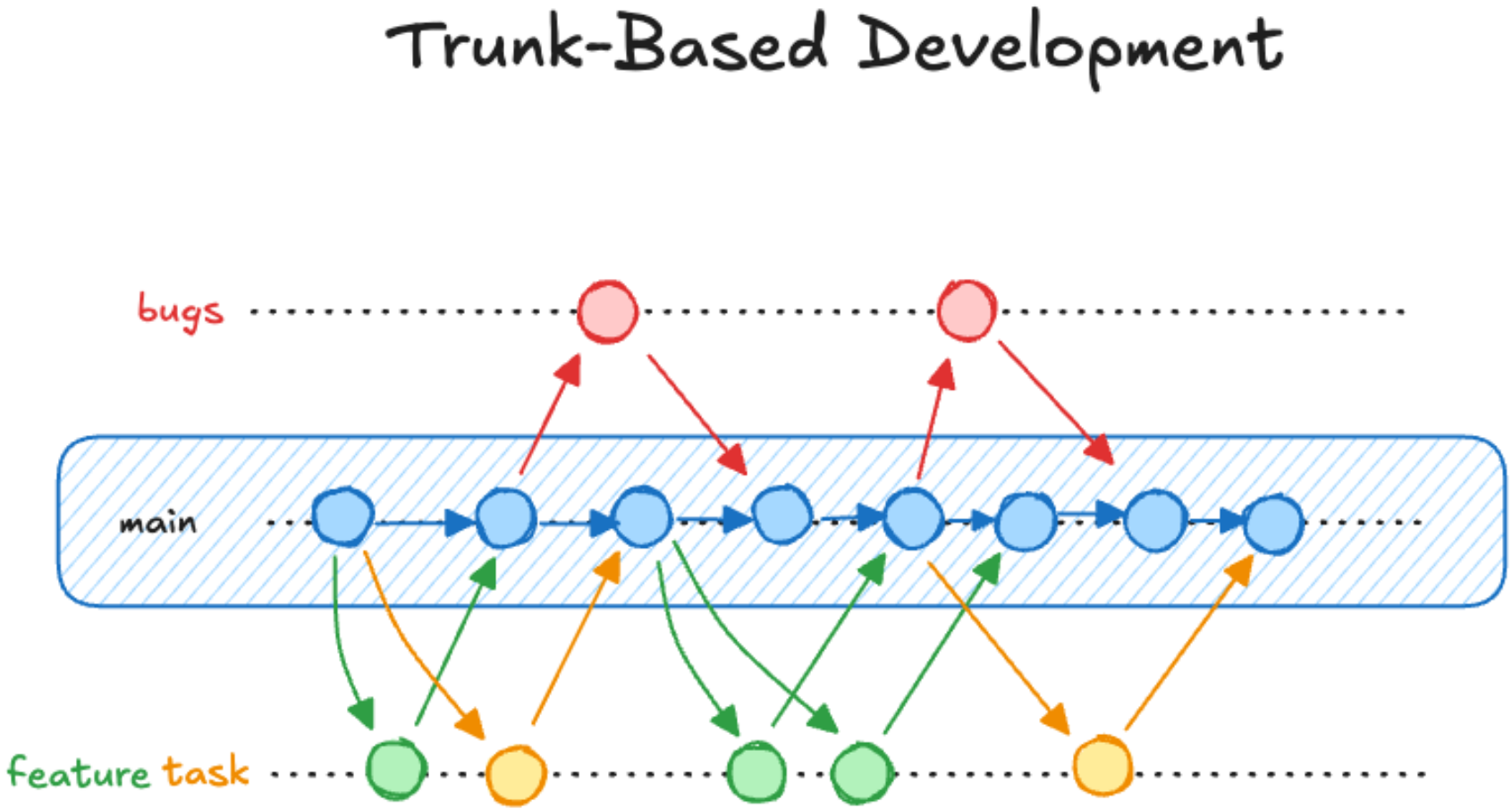
## Trabalho em Equipe e Concorrência

- **Sincronização Remota:** *fetch*, *pull*, *push* e vinculação com *remotes* (GitHub, GitLab, Bitbucket).
- **Branching e Merging:** Criação de ramificações independentes, integração de código (*merge* vs. *rebase*) e estratégias para resolução manual de conflitos.

CEUB

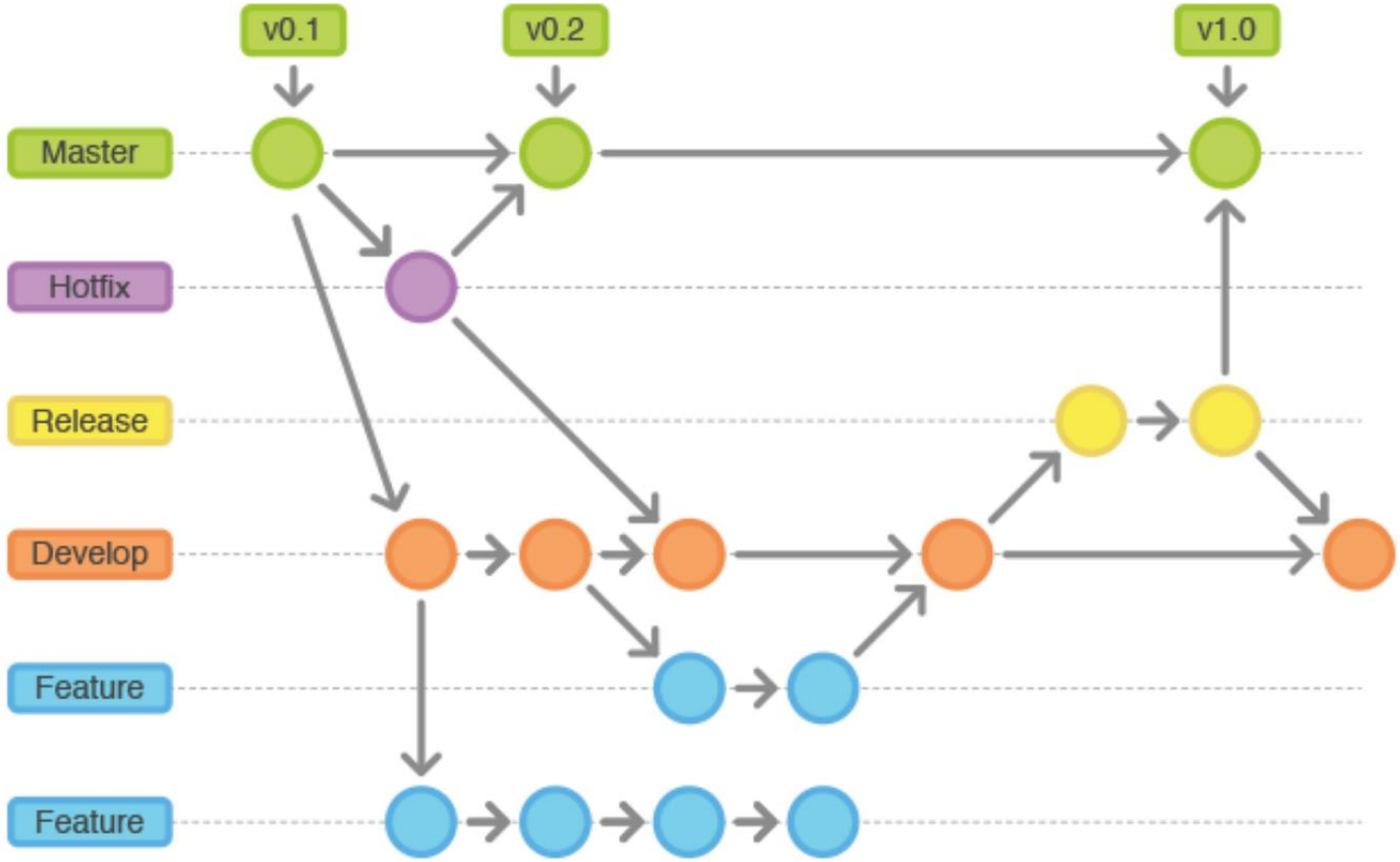
# Estratégias de Ramificação (*Branching Strategies*)

- **Trunk-Based Development:** Branches de curta duração e integração frequente (a abordagem mais alinhada a DevOps).



# Estratégias de Ramificação (*Branching Strategies*)

- **Gitflow:** Estrutura tradicional com branches dedicadas para *feature*, *develop*, *release* e *hotfix*.



# DevOps

## O Papel do Versionamento na Cultura DevOps

- **Gatilhos de Integração Contínua (CI):** Como commits e *merges* disparam automações de *build*, testes e análise estática.
- **GitOps:** O repositório Git como motor de implantação e controle de estado do ambiente de produção.
- **Infraestrutura como Código (IaC):** Versionamento de scripts de provisionamento (Terraform, Ansible, Kubernetes).

# Atividade prática

## Exercício de versionamento com Git

- Este exercício prático de *Git* ensina a criar um repositório local, fazer alterações, registrar um *commit*, enviar o projeto para o GitHub e convidar um colaborador para o projeto. Você vai usar o terminal e a interface do GitHub.

CEUB

CEUB

EDUCAÇÃO SUPERIOR